HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG — KHOA AN TOÀN THÔNG TIN

Đánh giá an toàn và bảo vệ dữ liệu người dùng ứng dụng OWASP Juice Shop v20.2.0

Học phần: DevOps và DevSecOps (SEC4349)

Thành viên	MSSV	Lỗi phụ trách
Nguyễn Toàn Thắng	B26CHAT039	F01 — SQL injection đăng nhập
Phạm Thị Minh Thúy	B26CHAT042	F02 — IDOR giỏ hàng
Đỗ Đức Quốc Anh	B26CHAT001	F03 — lộ endpoint /metrics
Hồ Anh Vũ	B26CHAT048	F04 — cấu hình an toàn container

Năm học 2026

Mục lục

Danh mục từ viết tắt

1. Giới thiệu và phạm vi

2. Ứng dụng Juice Shop và dữ liệu cần bảo vệ

2.1. Nghiệp vụ đăng ký và đăng nhập

2.2. Nghiệp vụ giỏ hàng và thanh toán

3. Môi trường triển khai và yêu cầu an toàn tối thiểu

3.1. Công cụ sử dụng

3.2. Chính sách và tiêu chuẩn đối chiếu

4. Bốn lỗi an toàn: phát hiện và xử lý

4.1. F01 — SQL injection tại đăng nhập

4.2. F02 — IDOR khi đọc giỏ hàng

4.3. F03 — Lộ endpoint /metrics không xác thực

4.4. F04 — Sai cấu hình an toàn của container

5. Policy as Code và Security as Code

5.1. Đoạn mã dò lỗi cốt lõi của mỗi demo

6. Đối chiếu chính sách bảo vệ dữ liệu

7. Kết luận

8. Tài liệu tham khảo

Phụ lục A — Mã nguồn chính sách Dockerfile (docker_security.rego)

Phụ lục B — Mã nguồn chính sách docker-compose (compose.rego)

Danh mục từ viết tắt

Bảng 1. Danh mục từ viết tắt

Viết tắt	Giải nghĩa
SQLi	SQL Injection — tiêm nhiễm câu lệnh SQL
IDOR	Insecure Direct Object Reference — tham chiếu đối tượng thiếu kiểm soát truy cập
JWT	JSON Web Token — token phiên
SAST	Phân tích tĩnh mã nguồn (Static Application Security Testing)
CI/CD	Tích hợp / triển khai liên tục
OPA	Open Policy Agent (ngôn ngữ Rego, công cụ Conftest)
GDPR	Quy định bảo vệ dữ liệu của EU (2016/679)
PCI-DSS	Chuẩn an toàn dữ liệu thẻ thanh toán
CIS	Center for Internet Security (CIS Docker Benchmark)
CWE / OWASP	Phân loại điểm yếu phần mềm / tổ chức an toàn ứng dụng web
UID	Mã định danh người dùng hệ điều hành

1. Giới thiệu và phạm vi

Báo cáo đánh giá an toàn ứng dụng **OWASP Juice Shop v20.2.0** theo hướng bảo vệ dữ liệu người dùng. Nhóm bốn thành viên, mỗi người phân tích và xử lý một lỗi an toàn: hai lỗi thuộc mã nguồn và hai lỗi thuộc triển khai/vận hành. Mỗi lỗi trình bày theo mạch **trước khi sửa → sau khi sửa**, gồm: cách phát hiện, cách xử lý, chính sách bị vi phạm, pha DevSecOps tương ứng và đoạn mã demo.

Để chứng minh, nhóm chạy cùng một kịch bản trên hai môi trường: **trước khi sửa** là một container chạy image gốc (127.0.0.1:3100), **sau khi sửa** là bản triển khai đã xử lý qua reverse proxy (127.0.0.1:3102). Bên trong, cổng 3000 là cổng mặc định của Juice Shop trong container; ứng dụng đã xử lý lắng nghe 3000 nhưng không mở ra máy chủ mà chỉ cho reverse proxy (cổng 3102) truy cập qua mạng nội bộ; cổng 3100 là container đối chứng chạy image gốc để so sánh. Juice Shop là ứng dụng cố ý chứa lỗ hổng để huấn luyện; các phát hiện nhằm minh họa quy trình, không phải đánh giá dự án gốc.

Bảng 2. Phân công thành viên

Thành viên	Lỗi phụ trách	Nhóm lỗi	Pha DevSecOps
Nguyễn Toàn Thắng — B26CHAT039	F01 SQL injection tại đăng nhập	Mã nguồn	Code, Test
Phạm Thị Minh Thúy — B26CHAT042	F02 IDOR khi đọc giỏ hàng	Mã nguồn (phân quyền)	Design, Code, Test
Đỗ Đức Quốc Anh — B26CHAT001	F03 Lộ endpoint /metrics không xác thực	Triển khai/vận hành	Deploy, Operate
Hồ Anh Vũ — B26CHAT048	F04 Sai cấu hình an toàn của container	Triển khai (build)	Build, Release

2. Ứng dụng Juice Shop và dữ liệu cần bảo vệ

Juice Shop là ứng dụng web thương mại điện tử mô phỏng (bán nước ép), do OWASP phát hành để huấn luyện an toàn ứng dụng. Ứng dụng là một Single Page Application với các thành phần sau:

Bảng 3. Thành phần của ứng dụng Juice Shop

Thành phần	Công nghệ	Vai trò
Giao diện người dùng	Angular (SPA)	Hiển thị, gọi REST API
Máy chủ ứng dụng	Node.js + Express	Xử lý REST API và nghiệp vụ
Truy xuất dữ liệu	ORM Sequelize trên SQLite	Lưu Users, Baskets, Cards…
Xác thực phiên	JWT (RS256), thư viện jsonwebtoken	Cấp và kiểm tra token đăng nhập
Giám sát	prom-client, endpoint `/metrics`	Xuất chỉ số dạng Prometheus
Đóng gói	Docker image distroless, chạy UID 65532	Triển khai bằng container

Các thành phần này đều liên quan tới bốn lỗi được phân tích: REST API/ORM (F01, F02), endpoint giám sát (F03) và cách đóng gói container (F04). Yêu cầu bảo vệ dữ liệu được xác định theo GDPR (nguyên tắc tại Điều 5, nghĩa vụ an toàn tại Điều 32) và PCI-DSS cho dữ liệu thẻ thanh toán; đồng thời đối chiếu pháp luật Việt Nam — **Nghị định 13/2023/NĐ-CP** về bảo vệ dữ liệu cá nhân và **Luật An toàn thông tin mạng 2015** (chi tiết ở mục 3.2 và 6).

Bảng 4. Dữ liệu cần bảo vệ và cơ sở chính sách

Dữ liệu	Yêu cầu bảo vệ	Cơ sở chính sách
Email, hồ sơ	Chỉ chủ tài khoản/quản trị truy cập	GDPR Điều 5(1)(f), 25(2)
Mật khẩu	Không để lộ; không bị vượt xác thực	GDPR Điều 32(1)(b)
Token phiên (JWT)	Chỉ cấp khi xác thực đúng	GDPR Điều 32(1)(b)
Giỏ hàng	Ràng buộc chủ sở hữu ở mọi truy vấn	GDPR Điều 5(1)(f), 25(2)
Thẻ thanh toán	Không lưu/không lộ số thẻ đầy đủ	PCI-DSS 3.4; GDPR Điều 32

2.1. Nghiệp vụ đăng ký và đăng nhập

Người dùng đăng ký qua `POST /api/Users` và đăng nhập qua `POST /rest/user/login`. Route `routes/login.ts` truy vấn bảng Users; nếu khớp thì cấp JWT và mã giỏ hàng. Ranh giới tin cậy quan trọng: email do client gửi đi thẳng vào câu lệnh SQL — đây là gốc của lỗi **F01**.

2.2. Nghiệp vụ giỏ hàng và thanh toán

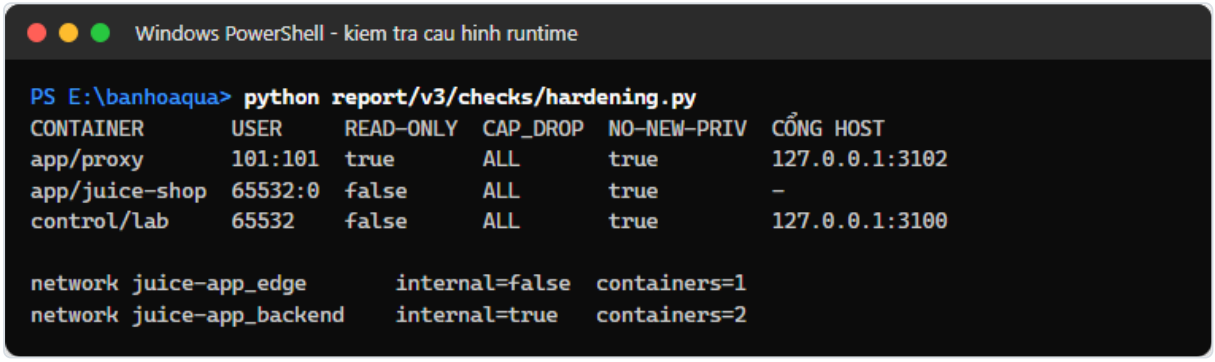
Sau khi đăng nhập, người dùng gọi `GET /rest/basket/:id` để xem giỏ. Route `routes/basket.ts` tìm giỏ theo `id` trên URL nhưng không kiểm tra giỏ có thuộc người gọi hay không — gốc của lỗi **F02**. Dữ liệu giỏ hàng phản ánh thói quen mua sắm của người dùng nên cần được bảo vệ theo quyền sở hữu.

3. Môi trường triển khai và yêu cầu an toàn tối thiểu

Ứng dụng chạy bằng Docker trên Windows (WSL2), triển khai bằng `docker-compose.yml`. Nhóm áp dụng các yêu cầu an toàn tối thiểu theo CIS Docker Benchmark và nguyên tắc giảm bề mặt tấn công.

Bảng 5. Kiểm soát an toàn tối thiểu

Kiểm soát	Cách thực hiện	Tiêu chuẩn
Giảm bề mặt mạng	Chỉ reverse proxy publish cổng, chỉ trên 127.0.0.1; backend ở mạng internal	GDPR Điều 32
Không chạy root	Juice Shop UID 65532, Nginx UID 101	CIS Docker 4.1
Đặc quyền tối thiểu	`cap_drop: ALL`, `no-new-privileges`, filesystem chỉ đọc nơi được	CIS Docker 5.3/5.12
Ghim phiên bản	Luật Rego chặn `:latest`, cảnh báo khi base image chưa ghim digest	CIS Docker 4.5
Lọc đường vào	Nginx chặn /metrics, giới hạn 5 lần đăng nhập/phút	GDPR Điều 32



Hình 1. Cấu hình an toàn của các container đang chạy: không chạy root, drop capability, chỉ proxy mở cổng loopback

Giới hạn còn lại: container đối chứng 3100 chạy image gốc chỉ dùng để so sánh trước/sau; HTTP nội bộ chưa dùng TLS; base image chưa ghim theo digest (luật Rego hiện mới cảnh báo, chưa bắt buộc trên image đối chứng).

3.1. Công cụ sử dụng

Bảng 6. Công cụ và vai trò

Công cụ	Loại	Vai trò trong báo cáo
Docker, Docker Compose	Nền tảng container	Triển khai ứng dụng và môi trường kiểm thử
Nginx	Reverse proxy	Chặn /metrics, giới hạn tốc độ đăng nhập (F03)
Semgrep	SAST (phân tích tĩnh)	Đọc mã nguồn (không chạy), chỉ ra dòng lỗi F01/F02 — bước *tìm ra* lỗi
Gitleaks	Quét secret	Tìm khóa/bí mật lộ trong mã nguồn
node:test (Supertest)	Kiểm thử động	Chạy ứng dụng thật, tái hiện lỗi *trước khi vá* (đỏ) và kiểm chứng *sau khi vá* (xanh)
Conftest / OPA (Rego)	Policy as Code	Kiểm tra Dockerfile và cấu hình triển khai (F03/F04)

Giới hạn của phân tích tĩnh. SAST (Semgrep) chỉ *tìm ra mẫu lỗi* trên mã nguồn, **không chứng minh được hành vi khi chạy** — ví dụ nó không biết một endpoint xoá dữ liệu có thực sự xoá hay chỉ ghi nhận yêu cầu, hay một API có trả về đầy đủ số thẻ hay không. Những điều đó phải do **kiểm thử động** (node:test) và **rà soát của con người** khẳng định. Vì vậy báo cáo kết hợp bốn lớp — SAST, kiểm thử động, Policy as Code (build-time) và Compliance as Code (runtime) — thay vì dựa vào một công cụ duy nhất.

3.2. Chính sách và tiêu chuẩn đối chiếu

Các phát hiện được đối chiếu với chính sách/tiêu chuẩn phổ biến, không phải quy ước tự đặt. Nhóm dùng **song song** khung quốc tế và pháp luật Việt Nam về bảo vệ dữ liệu cá nhân.

Quốc tế:

- **GDPR** (Regulation (EU) 2016/679): khung bảo vệ dữ liệu cá nhân; dùng các Điều 5 (nguyên tắc), 25 (bảo vệ ngay từ thiết kế) và 32 (an toàn khi xử lý) làm cơ sở đánh giá dữ liệu cần bảo vệ.
- **PCI-DSS v3.2.1**: chuẩn an toàn dữ liệu thẻ thanh toán; Yêu cầu 3.4 (không lưu/không lộ số thẻ đầy đủ), 8.3 (không nhúng bí mật), 2.2.x (không chạy root). *Báo cáo dùng nhất quán đánh số theo v3.2.1* — ở PCI-DSS v4.0 các số này thay đổi (ví dụ 3.4 → 3.5.1).
- **OWASP Top 10 2021 & CWE**: phân loại loại lỗi — A03 Injection/CWE-89 (F01), A01 Broken Access Control/CWE-639 (F02).

- **CIS Docker Benchmark**: chuẩn cấu hình container an toàn — các mục 4.1, 4.5, 4.9, 4.10, 5.8 dùng cho F04.
- **SLSA / OCI image-spec / NIST SP 800-190**: nguồn gốc, toàn vẹn image và an toàn container (F04).

Việt Nam:

- **Nghị định 13/2023/NĐ-CP** về bảo vệ dữ liệu cá nhân (hiệu lực 01/7/2023): **Điều 3** — 8 nguyên tắc (trong đó có nguyên tắc *bảo mật, an toàn*); **Điều 26** — biện pháp bảo vệ dữ liệu cá nhân gồm biện pháp *quản lý* và *kỹ thuật*, áp dụng ngay từ đầu và suốt quá trình xử lý; **Điều 27** — bảo vệ dữ liệu cá nhân cơ bản. Tương ứng GDPR Điều 5/25/32.
- **Luật An toàn thông tin mạng 2015 (86/2015/QH13)**: **Điều 16** — nguyên tắc bảo vệ thông tin cá nhân; **Điều 17** — không cung cấp/chia sẻ thông tin cá nhân cho bên thứ ba khi chưa được đồng ý (liên quan F02/F03); **Điều 19** — bảo đảm an toàn thông tin cá nhân trên mạng bằng biện pháp quản lý, kỹ thuật và *tuân thủ tiêu chuẩn, quy chuẩn kỹ thuật*.
- **TCVN 11930:2017** — tiêu chuẩn kỹ thuật quốc gia *Yêu cầu cơ bản về an toàn hệ thống thông tin theo cấp độ*, làm cơ sở kỹ thuật cho cấu hình triển khai tối thiểu (F03/F04).

4. Bốn lỗi an toàn: phát hiện và xử lý

4.1. F01 — SQL injection tại đăng nhập

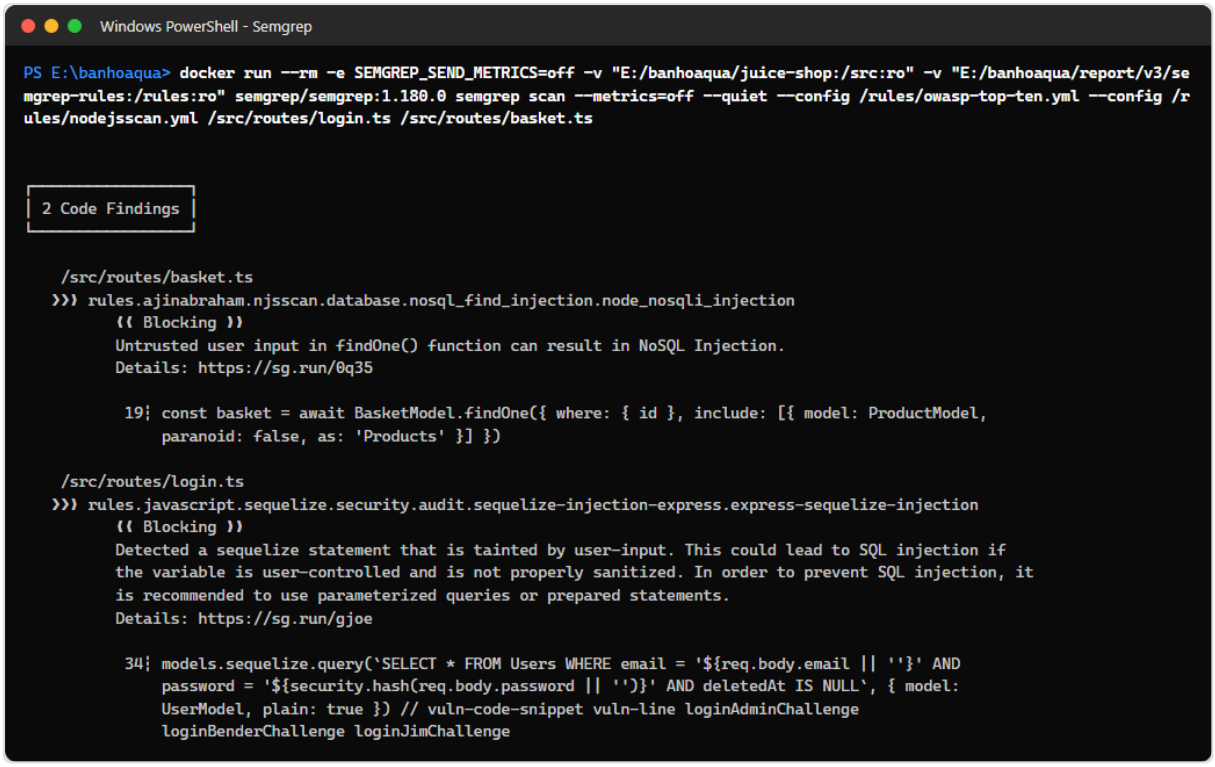
Trước khi sửa

Route `routes/login.ts` ghép thẳng email vào câu SQL:

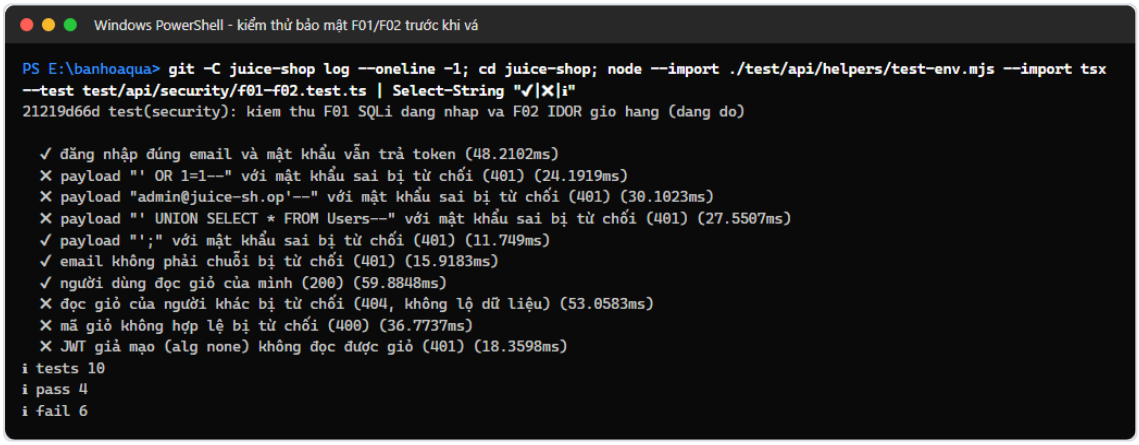
```
models.sequelize.query(`SELECT * FROM Users WHERE email = '${req.body.email || ''}'
  AND password = '${security.hash(req.body.password || '')}' AND deletedAt IS NULL`, {
model: UserModel, plain: true })
```

Với email `' OR 1=1--`, mệnh đề WHERE luôn đúng và phần kiểm tra mật khẩu bị biến thành chú thích, nên máy chủ cấp token của tài khoản đầu tiên (quản trị) mà không cần mật khẩu. Đây là CWE-89, OWASP A03:2021.

Phát hiện. Semgrep (luật `express-sequelize-injection`) chỉ đúng dòng lỗi; kiểm thử `test/api/security/f01-f02.test.ts` gửi payload kèm mật khẩu sai và yêu cầu nhận 401 — trên mã chưa sửa, test đỏ (@fig(f0102-red)). Demo `demos/tv1_f01_sqli.py` xác nhận token được cấp (@fig(tv1)).



Hình 2. Semgrep phát hiện truy vấn nhiễm dữ liệu người dùng tại login.ts và basket.ts



Hình 3. Kiểm thử bảo mật F01/F02 trên mã chưa sửa: các ca kiểm thử đỏ ứng với lỗ hổng còn tồn tại

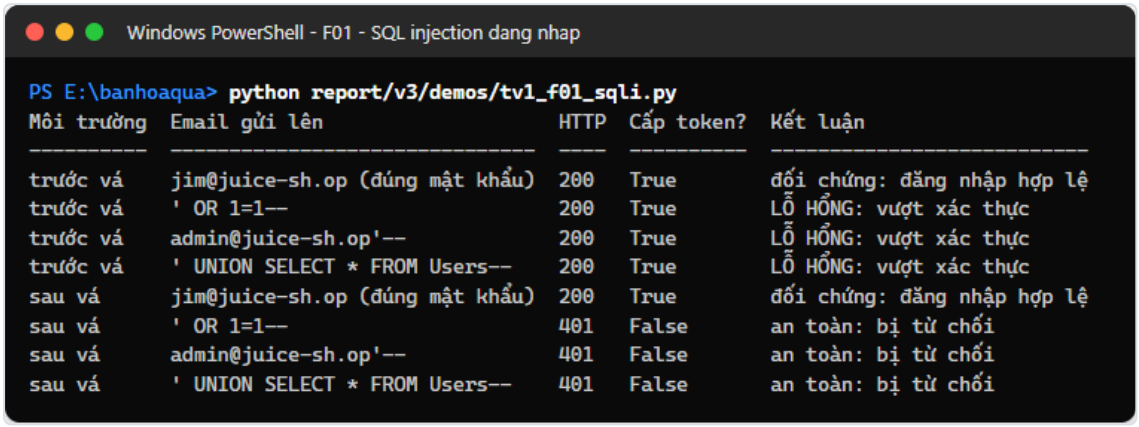
Sau khi sửa

Dùng truy vấn tham số (bind parameter):

```
const email = typeof req.body.email === 'string' ? req.body.email : ''
const password = typeof req.body.password === 'string' ? req.body.password : ''
models.sequelize.query('SELECT * FROM Users WHERE email = $email AND password = $password
AND deletedAt IS NULL',
  { bind: { email, password: security.hash(password) }, model: UserModel, plain: true })
```

Câu lệnh được biên dịch trước, giá trị gửi riêng nên ký tự SQL trong email chỉ còn là dữ liệu. Sau khi sửa, kiểm thử bảo mật xanh toàn bộ (@fig(f0102-green)) và demo cho thấy mọi payload đều bị từ chối 401 trong khi đăng nhập hợp lệ vẫn hoạt động (@fig(tv1)).

Giới hạn còn lại: bản vá chỉ xử lý injection; cùng dòng code vẫn gọi `security.hash` của Juice Shop — hàm này băm mật khẩu bằng **MD5 không muối** (`lib/insecurity.ts`, CWE-327 / OWASP A02:2021). Đây là lỗ hổng lưu trữ mật khẩu nằm ngoài phạm vi F01 (vốn chỉ chặn vượt xác thực); khi triển khai thật cần thay bằng bcrypt/argon2 có muối.



Hình 4. Demo F01: cùng payload, trước khi sửa được cấp token, sau khi sửa bị từ chối

Chính sách vi phạm: GDPR Điều 32(1)(b) (bảo mật khi xử lý dữ liệu); Nghị định 13/2023 Điều 26 (biện pháp kỹ thuật bảo vệ dữ liệu cá nhân); Luật ATTTM 2015 Điều 19. **Pha & Security as Code:** phát hiện ở pha Code (SAST), chặn tái diễn ở pha Test bằng kiểm thử bảo mật trong pipeline.

4.2. F02 — IDOR khi đọc giỏ hàng

Trước khi sửa

Route `routes/basket.ts` tìm giỏ chỉ theo mã trên URL:

```
const basket = await BasketModel.findOne({ where: { id }, include: [{ model: ProductModel,
as: 'Products' }] })
```

Middleware chỉ kiểm tra token hợp lệ, không kiểm tra quyền sở hữu, nên đổi `/rest/basket/8` thành `/rest/basket/9` là đọc được giỏ người khác (CWE-639, OWASP A01:2021).

Phát hiện. Kiểm thử hai tài khoản A, B: dùng token của A đọc giỏ của B. Trên mã chưa sửa trả về đúng giỏ của B (@fig(tv2)); các ca tương ứng trong `f01-f02.test.ts` đỏ.

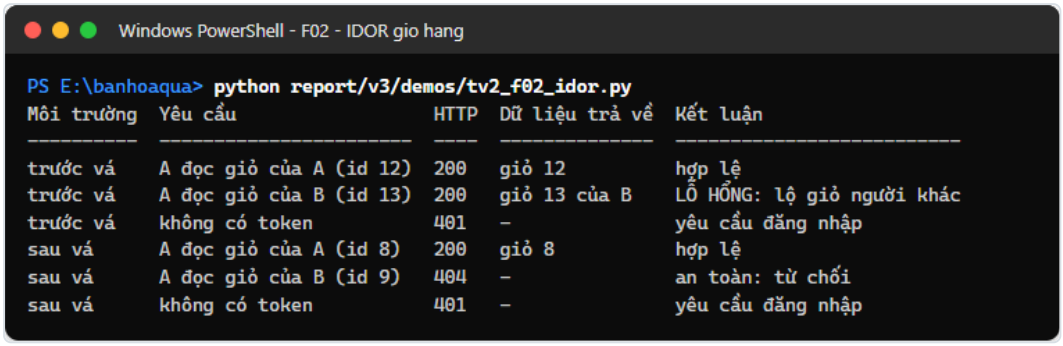
Sau khi sửa

Ràng buộc chủ sở hữu ngay trong truy vấn, danh tính lấy từ phiên do máy chủ cấp:

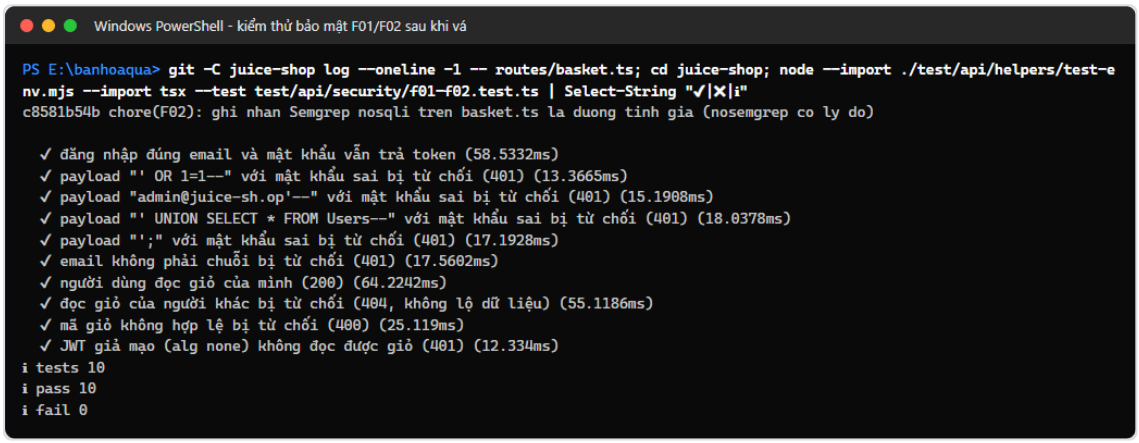
```
const user = security.authenticatedUsers.from(req)
if (!user?.data?.id) return res.status(401).end()
const id = Number(req.params.id)
if (!Number.isSafeInteger(id) || id <= 0) return res.status(400).end()
```

```
const basket = await BasketModel.findOne({ where: { id, UserId: user.data.id }, include: [{
model: ProductModel, as: 'Products' }] })
if (!basket) return res.status(404).end()
```

Sau khi sửa, A đọc giỏ của B nhận 404, giỏ của chính mình vẫn 200 (@fig(tv2)).



Hình 5. Demo F02: trước khi sửa A đọc được giỏ của B, sau khi sửa nhận 404



Hình 6. Kiểm thử bảo mật F01/F02 sau khi sửa: toàn bộ ca đạt

Chính sách vi phạm: GDPR Điều 5(1)(f), 25(2); Nghị định 13/2023 Điều 3 (nguyên tắc an toàn) và Điều 26; Luật ATTTM 2015 Điều 17 (không để lộ dữ liệu cá nhân cho bên thứ ba). **Pha & Security as Code:** phát hiện ở Test (kiểm thử phân quyền), giữ trong pipeline để chống tái diễn.

4.3. F03 — Lộ endpoint /metrics không xác thực

Trước khi sửa

Juice Shop mở `/metrics` (Prometheus) không cần đăng nhập, lộ số người dùng, số đơn hàng, tổng số dư ví. Khi container publish thẳng cổng ứng dụng, ai cũng gọi được từ host. Đây là lỗi **triển khai/vận hành**.

Phát hiện. Demo `demos/tv3_f03_metrics.py` gọi `/metrics` từ host: trước khi sửa trả 200 với hàng trăm dòng số liệu (@fig(tv3)).

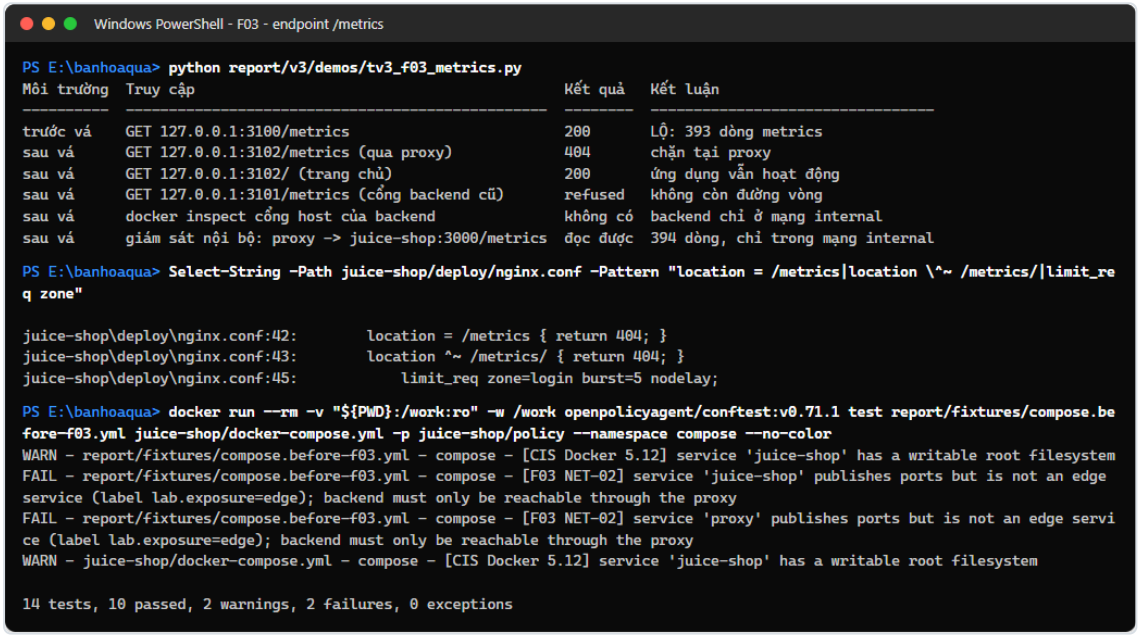
Sau khi sửa

Bỏ cổng host của backend, chỉ cho reverse proxy mở cổng loopback; proxy chặn `/metrics` và giới hạn tốc độ đăng nhập:

```
location = /metrics { return 404; }
location = /rest/user/login { limit_req zone=login burst=5 nodelay; limit_req_status 429;
proxy_pass http://juice-shop:3000; }
```

```
juice-shop:
  networks: [backend]            # không còn khóa ports -> không truy cập trực tiếp từ host
networks:
  backend: { internal: true }
```

Sau khi sửa: `/metrics` qua proxy trả 404 và cổng backend cũ bị từ chối kết nối **từ host** — tức chặn ở lối vào từ bên ngoài, không phải đóng tuyệt đối: trong mạng internal, hệ thống giám sát vẫn đọc được metrics đúng mục đích (@fig(tv3)).



Hình 7. Demo F03: /metrics lộ khi truy cập trực tiếp, bị chặn sau khi sửa

Chính sách vi phạm: GDPR Điều 25(2), 32(1)(b); Nghị định 13/2023 Điều 26; Luật ATTTM 2015 Điều 17, 19. **Pha & Policy as Code:** lỗi ở Deploy/Operate; luật `compose.rego` chặn mọi cấu hình cho backend mở cổng (mục 5).

4.4. F04 — Sai cấu hình an toàn của container

Trước khi sửa

Image đóng gói Juice Shop có thể vi phạm chính sách an toàn tối thiểu mà vẫn bị bỏ sót nếu kiểm tra sai chỗ. Một `Dockerfile.vulnerable` minh họa bốn vi phạm:

```
FROM node:22-alpine
COPY . .
```

```
ENV API_TOKEN=supersecret123  # nhúng bí mật vào image (lộ qua docker history)
EXPOSE 3000 22 3306           # mở cổng SSH 22, DB 3306
CMD ["npm", "start"]           # không USER (chạy root), không nhãn nguồn gốc
```

Các kiểm tra ngây thơ dễ bị qua mặt: "có lệnh USER" bỏ qua Dockerfile nhiều stage; "EXPOSE không chứa cổng cấm" vô nghĩa vì EXPOSE chỉ là tài liệu; nhãn `mfa="true"` tự khai được.

Rủi ro nếu để nguyên, không vá. Mỗi vi phạm dẫn tới một rủi ro an toàn cụ thể:

- **Chạy bằng root (LEAST-PRIVILEGE):** khi ứng dụng bị chiếm quyền qua một lỗ hổng khác — ví dụ chính F01 — tiến trình độc hại có ngay quyền root *bên trong* container, dễ ghi đè file hệ thống, cài công cụ, và nếu kết hợp một lỗ hổng nhân hệ điều hành thì **thoát container, leo thang đặc quyền** lên máy chủ. Chạy non-root (UID 65532) giới hạn thiệt hại ngay cả khi ứng dụng bị khai thác.
- **Mở cổng quản trị/CSDL (ATTACK-SURFACE):** khai báo EXPOSE 22 (SSH), 3306 (MySQL) báo hiệu các dịch vụ này có thể được bật, **mở rộng bề mặt tấn công** và tạo đường dò quét, đăng nhập vét cạn vào dịch vụ quản trị và cơ sở dữ liệu — nơi chứa dữ liệu người dùng.
- **Rủi ro chuỗi cung ứng (SUPPLY-CHAIN):** thiếu nhãn nguồn gốc `org.opencontainers.image.source`/`revision` thì khi có sự cố **không truy vết được image bắt nguồn từ commit nào** để điều tra, thu hồi (đây là *provenance khai báo* theo SLSA L1 — phục vụ truy vết; muốn chắc chắn hơn thì ký *attestation*). Ngoài ra, dùng `:latest` hay chưa ghim digest khiến image có thể **bị đổi âm thầm** giữa các lần build, bản chạy thật khác bản đã kiểm thử; lệnh `ADD` có thể kéo nội dung từ xa không kiểm soát. Ghim digest, gắn nhãn nguồn gốc và dùng COPY giúp image **truy vết được, tái lập được và bất biến**.
- **Bí mật nhúng trong image (SECRET-IN-IMAGE):** đặt mật khẩu/token trong `ENV`/`ARG` làm bí mật **bị bake vào layer của ảnh** — bất kỳ ai `docker pull` rồi chạy `docker history` đều đọc được, kể cả khi dòng đó đã bị xóa ở layer sau. Thay vào đó dùng *build secret* hoặc nạp biến môi trường lúc chạy. (Đây là **luật riêng nhóm bổ sung**, không có trong bản mẫu của lab.)

Sau khi sửa

Việc xử lý gồm **hai phần tách bạch**: (1) **sửa chính file triển khai** cho đúng cấu hình an toàn, và (2) **viết chính sách kiểm tra tự động** để chống tái diễn. Cần nói rõ: **Conftest không tự sửa file** — nó chỉ là *cổng kiểm tra*, đọc Dockerfile và trả PASS/FAIL kèm lý do; nếu còn vi phạm thì chặn phát hành. Người viết Dockerfile mới là người sửa.

(1) Sửa file Dockerfile triển khai. So với bản lỗi, bản vá (`report/fixtures/Dockerfile.remediated`) sửa các điểm tương ứng từng vi phạm:

```
FROM node:22-alpine
# THÊM nhãn nguồn gốc OCI  -> đạt SUPPLY-CHAIN (provenance)
LABEL org.opencontainers.image.source="https://github.com/juice-shop/juice-shop" \
      org.opencontainers.image.revision="${VCS_REF}"
RUN addgroup -S -g 10001 juicers && adduser -S -u 10001 -G juicers juicers
COPY --chown=10001:0 . .
USER 10001:10001          # THÊM USER non-root       -> đạt LEAST-PRIVILEGE
EXPOSE 3000               # BỎ cổng 22 và 3306        -> đạt ATTACK-SURFACE
CMD ["npm", "start"]
```

Tức: thêm lệnh `USER` chạy non-root, bỏ hai cổng quản trị/CSDL khỏi `EXPOSE`, thêm nhãn nguồn gốc OCI, và **không đặt bí mật trong `ENV`/`ARG`** (dùng *build secret* hoặc nạp biến môi trường lúc chạy). Khi phát hành thật nên ghim base image theo digest (hiện luật Rego mới cảnh báo, chưa chặn). Image triển khai thực tế của nhóm (`deploy/Dockerfile.hotfix`) và cả Dockerfile gốc của Juice Shop đều đã thỏa các điều này.

(2) Viết chính sách thành mã (Policy as Code) bằng Rego/Conftest (`juice-shop/policy/docker_security.rego`), đặt mỗi luật đúng điểm thực thi để không bị qua mặt. Mã nguồn đầy đủ của bộ luật ở **Phụ lục A**.

Cần lưu ý Conftest là **bộ máy chính sách tổng quát, không có luật sẵn**: bản thân nó không tự quét ra lỗi nào. Chính **bộ luật Rego nhóm viết** (kèm mười một unit test chứng minh luật bắt đúng và không bị qua mặt) mới là *cách phát hiện* của F04 — đúng tinh thần Policy as Code: tổ chức tự mã hóa chính sách của mình thành mã, kiểm chứng và chạy lại được trong CI.

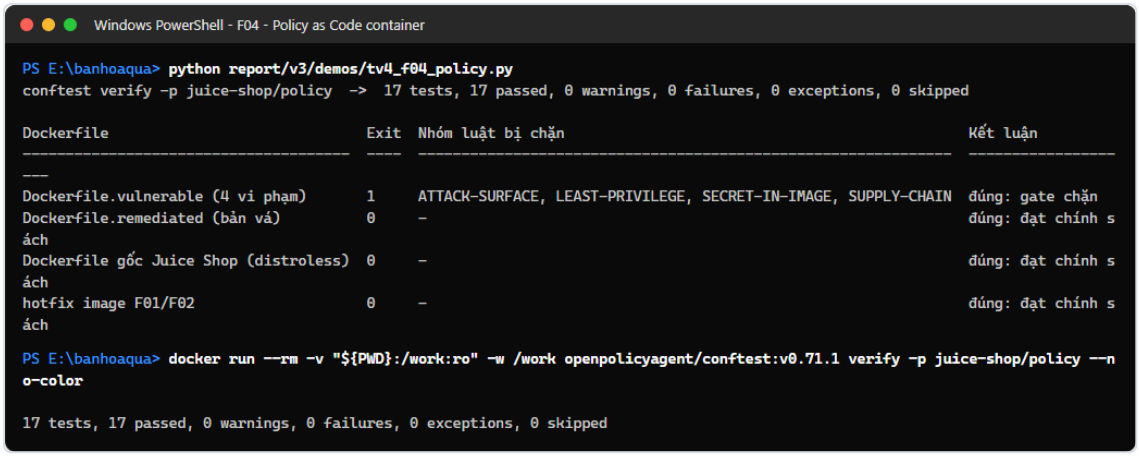
Các luật và tiêu chuẩn đối chiếu:

Bảng 7. Luật kiểm tra cấu hình container và tiêu chuẩn đối chiếu

Luật	Vi phạm bị chặn	Tiêu chuẩn (quốc tế + Việt Nam)
LEAST-PRIVILEGE	stage cuối chạy root	CIS Docker 4.1; PCI-DSS 2.2.6; **NĐ 13/2023 Đ.26; Luật ATTTM Đ.19**
ATTACK-SURFACE	EXPOSE cổng 22/23/3306/5432	CIS Docker 5.8; NIST SP 800-190; **Luật ATTTM Đ.19**
SUPPLY-CHAIN	thiếu nhãn nguồn gốc OCI (provenance khai báo); dùng :latest/ADD; chưa ghim digest	OCI image-spec; SLSA; CIS Docker 4.5/4.9
SECRET-IN-IMAGE *(nhóm bổ sung)*	nhúng bí mật trong ENV/ARG	CIS Docker 4.10; PCI-DSS 8.3; GDPR Đ.32; **NĐ 13/2023 Đ.26**

Nội dung một luật và cách trích chuẩn. Mỗi luật là một mệnh đề Rego: phần *thân* nêu điều kiện vi phạm trên cây cú pháp của Dockerfile/compose (ví dụ "stage cuối không có lệnh USER", "EXPOSE chứa cổng 22", "ENV chứa từ khóa bí mật"), phần *thông điệp* (`msg`) là lý do trả về khi vi phạm. **Cách trích dẫn chuẩn** được nhúng thẳng vào `msg`: mỗi thông điệp kết thúc bằng danh sách điều khoản áp dụng — ví dụ luật LEAST-PRIVILEGE trả về *"...container runs as root. (CIS Docker 4.1; PCI-DSS 2.2.6; NĐ 13/2023 Đ.26; Luật ATTTM 2015 Đ.19)"*. Nhờ vậy khi cổng chặn phát hành, người đọc thấy ngay **luật nào bị vi phạm và nó dựa trên điều khoản nào của chuẩn nào** — đây chính là cách policy "trích" chuẩn thành mã, kiểm chứng lại được bằng `conftest`. Mã đầy đủ (kèm trích dẫn) ở Phụ lục A/B.

`conftest verify` cho toàn bộ chính sách đạt 17/17 ca kiểm thử (11 ca cho chính sách Dockerfile của F04, 6 ca cho chính sách compose của F03). Kết quả: `Dockerfile.vulnerable` bị chặn 4 nhóm luật (LEAST-PRIVILEGE, ATTACK-SURFACE, SUPPLY-CHAIN, SECRET-IN-IMAGE), bản vá đạt, và Dockerfile gốc distroless của Juice Shop **đạt toàn bộ chính sách an toàn** (@fig(tv4)).



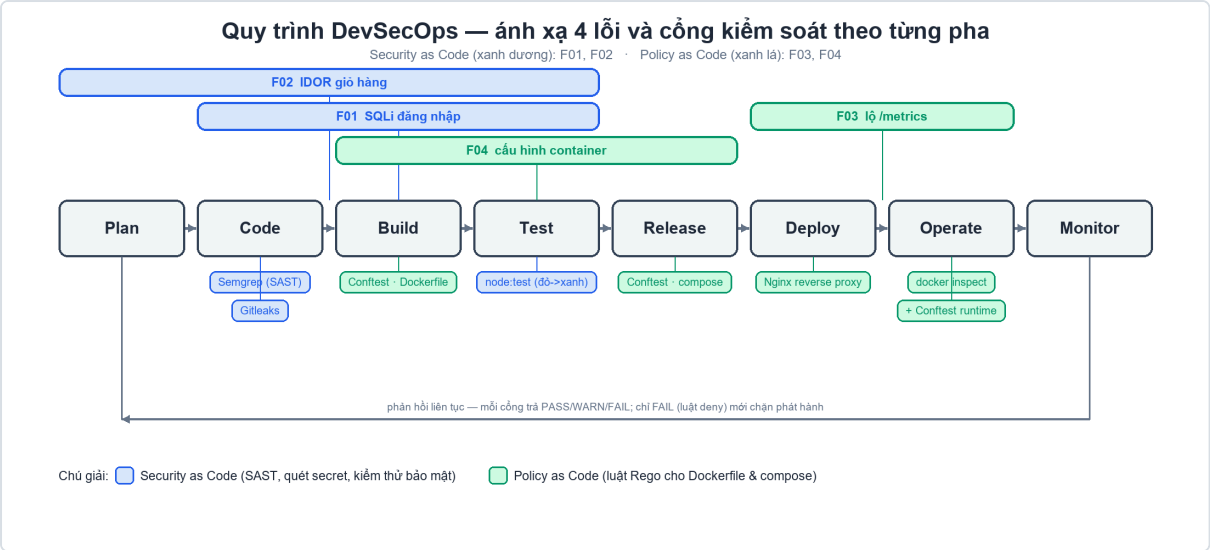
Hình 8. Demo F04: Dockerfile.vulnerable bị chặn bốn nhóm luật, bản vá và Dockerfile gốc đạt chính sách

Phân định nguồn. Bộ luật `docker_security.rego` **kế thừa** cấu trúc và bốn luật cốt lõi từ Appendix A.1 của tài liệu lab (LEAST-PRIVILEGE theo USER ở stage cuối, ATTACK-SURFACE theo cổng cấm, nhãn provenance OCI, SUPPLY-CHAIN `:latest`/digest) cùng cách viết unit test. **Phần nhóm tự làm** gồm: (a) bỏ luật HEALTHCHECK/OBSERVABILITY của lab (thiên về vận hành hơn an toàn, và distroless không chạy được); (b) gộp provenance vào nhóm SUPPLY-CHAIN và nêu rõ đây là provenance *khai báo* (SLSA L1); (c) **tự viết luật** `SECRET-IN-IMAGE` chặn bí mật nhúng trong ENV/ARG — luật này **không có trong lab**, kèm hai unit test. Riêng chính sách `compose.rego` (F03) là **do nhóm tự viết cho docker-compose**: lab chỉ enforce trên manifest Kubernetes (k8s.rego), nhóm chuyển các ý tưởng least-privilege/firewall/secret sang đúng định dạng triển khai thật của mình. Mã nguồn đầy đủ ở **Phụ lục A** (Dockerfile) và **Phụ lục B** (compose).

Chính sách vi phạm: GDPR Điều 25(1), 32(1)(d); CIS Docker Benchmark 4.1; Nghị định 13/2023 Điều 26; Luật ATTTM 2015 Điều 19; TCVN 11930:2017. **Pha & Policy as Code:** lỗi ở Build/Release; chính sách chạy như một cổng kiểm tra trong pipeline.

5. Policy as Code và Security as Code

Toàn bộ việc phát hiện và chống tái diễn bốn lỗi được viết thành mã và chạy tự động như một chuỗi kiểm soát (pipeline cục bộ), đúng tinh thần DevSecOps. Hình dưới ánh xạ từng lỗi và cổng kiểm soát vào các pha của vòng đời DevSecOps (xanh dương = Security as Code cho F01/F02; xanh lá = Policy as Code cho F03/F04):



Hình 9. Quy trình DevSecOps: ánh xạ bốn lỗi F01–F04 và các cổng kiểm soát theo từng pha; Security as Code cho F01/F02, Policy as Code cho F03/F04

Hai nhóm kiểm soát trong hình:

- **Security as Code:** quét secret (Gitleaks), SAST (Semgrep) cho F01/F02, và kiểm thử bảo mật `f01-f02.test.ts` (đỏ trước vá, xanh sau vá).
- **Policy as Code (build-time):** luật Rego cho Dockerfile (F04) và cho `docker-compose` (F03) — chặn cấu hình mở cổng backend hay chạy root **trước khi triển khai**.
- **Compliance as Code (runtime):** kiểm tra **container đang chạy** bằng `docker inspect` + luật Rego runtime (`report/policy/runtime.rego`, `checks/hardening.py`) — xác nhận non-root, drop capability, chỉ proxy mở cổng loopback. So sánh container đối chứng (trước vá) với bản triển khai (sau vá) chính là cặp ca *lỗi cố ý / đạt* của kiểm tra tuân thủ.

Các kiểm soát này được viết thành mã, chạy lặp lại được — hiện chạy cục bộ (xem `DEMO.md`), và có thể ghép thành một chuỗi cổng (gate) trong CI. Mỗi cổng trả PASS/WARN/FAIL; chỉ mức **FAIL (luật `deny`)** mới chặn phát hành, còn mức **WARN (luật `warn`)** chỉ cảnh báo, không chặn. Trong `compose.rego`, các luật `cap_drop`/`read_only` là mức WARN (khuyến cáo), còn các luật mở cổng backend, bind ngoài loopback, privileged, mount docker.sock, secret nội tuyến là mức FAIL (chặn).

Bảng 8. Các cổng kiểm soát và lệnh chạy

Cổng kiểm soát	Công cụ	Lỗi được kiểm soát	Chặn phát hành
Quét secret	Gitleaks	khóa/bí mật lộ trong mã	có
SAST	Semgrep	F01, F02	có
Kiểm thử bảo mật	node:test (Supertest)	F01, F02 (đỏ→xanh)	có
Policy as Code — build	Conftest/OPA (Rego)	F03, F04 (Dockerfile, compose)	có — luật `deny`; luật `warn` chỉ cảnh báo
Compliance as Code — runtime	Conftest + `docker inspect`	F03, F04 (container đang chạy)	cảnh báo

Lệnh chạy cụ thể cho từng cổng và từng demo được ghi trong `DEMO.md` (thư mục gốc) và `report/HUONG_DAN_DEMO.md`.

Đoạn mã demo của mỗi thành viên không tách rời tài liệu môn học mà **áp dụng trực tiếp** kỹ thuật được dạy trong đó:

Bảng 9. Liên hệ đoạn mã demo với tài liệu môn học

Thành viên	Đoạn mã demo	Kỹ thuật & tài liệu môn học áp dụng
Nguyễn Toàn Thắng (F01)	`demos/tv1_f01_sqli.py`, kiểm thử `f01-f02.test.ts`	Shift-left, SAST, Test-Driven Security — slide kiến trúc DevSecOps
Phạm Thị Minh Thúy (F02)	`demos/tv2_f02_idor.py`, kiểm thử phân quyền	Security as Code, kiểm thử trong CI — slide DevSecOps
Đỗ Đức Quốc Anh (F03)	`demos/tv3_f03_metrics.py`, `nginx.conf`	OpSec giảm phơi nhiễm endpoint, reverse proxy & giới hạn tốc độ — slide OpSec/DevSecOps
Hồ Anh Vũ (F04)	`demos/tv4_f04_policy.py`, `docker_security.rego`	Policy as Code với Conftest — hướng dẫn Policy as Code (Conftest)

Mỗi thành viên lấy một kỹ thuật trong tài liệu môn học và hiện thực hóa thành đoạn mã chạy được để phát hiện hoặc chặn lỗi của mình.

5.1. Đoạn mã dò lỗi cốt lõi của mỗi demo

Mỗi script chạy bằng **một lệnh** (`python report/demos/<tên>.py`); phần dò lỗi cốt lõi trích dưới đây.

F01 — `tv1_f01_sqli.py`: gửi payload SQL kèm mật khẩu *sai*; còn lỗ hổng nếu vẫn được cấp token.

```
status, body = call(base, 'POST', '/rest/user/login', {'email': "' OR 1=1--", 'password':
'wrong-password'})
token = body.get('authentication', {}).get('token')   # có token = vượt xác thực = còn lỗ
hổng
```

F02 — `tv2_f02_idor.py`: dùng token của A đọc giỏ của B; lộ dữ liệu nếu trả đúng giỏ của B.

```
s, body = call(base, 'GET', f"/rest/basket/{b['bid']}", token=a['token'])
leaked = s == 200 and body['data']['id'] == b['bid']   # đọc được giỏ người khác = IDOR
```

F03 — `tv3_f03_metrics.py`: gọi `/metrics`; trước vá 200 (lộ), sau vá qua proxy trả 404.

```
s, body = call(BEFORE[1], 'GET', '/metrics')   # trước vá (3100): 200 + hàng trăm dòng số
liệu
s, _ = call(AFTER[1], 'GET', '/metrics')        # sau vá, qua proxy (3102): 404
```

F04 — `tv4_f04_policy.py`: chạy Conftest trên Dockerfile; thoát khác 0 nghĩa là gate chặn.

```
p = sh(*CF, 'test', target, '--parser', 'dockerfile', '-p', 'policy', '--namespace',
'dockerfile')
# p.returncode != 0 -> gate chặn; mỗi dòng FAIL là một luật Rego bị vi phạm
```

6. Đối chiếu chính sách bảo vệ dữ liệu

Nhóm đối chiếu mỗi lỗi với cả khung **quốc tế (GDPR)** và **pháp luật Việt Nam** — Nghị định 13/2023/NĐ-CP (bảo vệ dữ liệu cá nhân) và Luật An toàn thông tin mạng 2015.

Bảng 10. Ánh xạ lỗi với chính sách (quốc tế và Việt Nam)

Lỗi	GDPR	Nghị định 13/2023	Luật ATTTM 2015
F01 SQL injection	Điều 5(1)(f), 32(1)(b)	Điều 26	Điều 19
F02 IDOR	Điều 5(1)(f), 25(2)	Điều 3, 26	Điều 17, 19
F03 /metrics	Điều 25(2), 32(1)(b)	Điều 26	Điều 17, 19
F04 cấu hình container	Điều 25(1), 32(1)(d)	Điều 26	Điều 19

Đây là ánh xạ rủi ro kỹ thuật, không phải kết luận pháp lý. Riêng F04, cấu hình triển khai còn đối chiếu TCVN 11930:2017 (yêu cầu an toàn hệ thống thông tin theo cấp độ).

7. Kết luận

Bốn lỗi an toàn (hai lỗi mã nguồn F01/F02, hai lỗi triển khai F03/F04) đều được phát hiện bằng công cụ hoặc kiểm thử, xử lý và kiểm chứng theo mạch trước→sau, kèm đoạn mã demo của từng thành viên. Việc phát hiện và chống tái diễn được thể hiện dưới dạng Policy as Code và Security as Code trong một chuỗi kiểm soát DevSecOps. Các biện pháp này không có sẵn trong Juice Shop gốc (ứng dụng cố ý để lỗ hổng, không có truy vấn tham số, không reverse proxy/giới hạn tốc độ, không chính sách cấu hình container, không cổng kiểm soát trong CI) — đó là phần giá trị mà báo cáo bổ sung. Hướng dẫn tái lập nằm trong `report/HUONG_DAN_DEMO.md`.

8. Tài liệu tham khảo

- OWASP Juice Shop — https://owasp.org/www-project-juice-shop/
- GDPR, Regulation (EU) 2016/679, Điều 5, 25, 32.
- Pháp luật Việt Nam: Nghị định 13/2023/NĐ-CP về bảo vệ dữ liệu cá nhân (Điều 3, 26, 27); Luật An toàn thông tin mạng 2015 (86/2015/QH13, Điều 16, 17, 19); TCVN 11930:2017.
- CIS Docker Benchmark; PCI-DSS v3.2.1 (Yêu cầu 3.4, 8.3, 2.2.x); OWASP Top 10 2021; SLSA.
- Công cụ: Semgrep, Gitleaks, node:test (Supertest), Conftest/OPA.
- Tài liệu môn học: bộ slide DevSecOps và hướng dẫn Policy as Code (Conftest).

Phụ lục A — Mã nguồn chính sách Dockerfile (docker_security.rego)

Nguồn: kế thừa cấu trúc và các luật cốt lõi từ Appendix A.1 của tài liệu lab (`Policy_as_code_Lab ConfTest.docx`); phần nhóm điều chỉnh/bổ sung: bỏ luật HEALTHCHECK, gộp provenance vào SUPPLY-CHAIN, và **tự viết luật** **`SECRET-IN-IMAGE`** (xem mục 4.4). Namespace `dockerfile`, 8 luật `deny`; bộ kiểm thử `docker_security_test.rego` có 11 ca. Mỗi thông báo vi phạm trích kèm điều khoản chuẩn quốc tế và Việt Nam (NĐ 13/2023 Đ.26; Luật ATTTM 2015 Đ.19). Chạy: `conftest test <Dockerfile> -p juice-shop/policy --namespace dockerfile`.

```
# Policy as Code cho Dockerfile của Juice Shop — bám theo lab môn học "Policy as Code with
Conftest".
# Mỗi luật đặt tại điểm thực thi đúng thay vì các "original check" dễ bị qua mặt:
#   Least privilege  -> USER ở STAGE CUỐI, UID số, không phải root (thay vì "có USER bất kỳ
trong file")
#   Firewall/RBAC    -> cổng quản trị/DB không được EXPOSE (giảm bề mặt tấn công)
#   Supply chain     -> provenance khai báo (nhãn OCI source+revision, SLSA L1) + không
:latest, không ADD, cảnh báo base chưa ghim digest
#   Secret hygiene   -> KHÔNG nhúng bí mật (ENV/ARG) vào image (cải tiến của nhóm, ngoài
bản mẫu lab)
# Mỗi thông báo vi phạm trích sẵn điều khoản chuẩn áp dụng — quốc tế (CIS Docker, PCI-DSS,
NIST, OCI/SLSA)
# và Việt Nam (Nghị định 13/2023/NĐ-CP Điều 26 — biện pháp kỹ thuật bảo vệ dữ liệu cá nhân;
Luật An toàn
# thông tin mạng 2015 Điều 19 — biện pháp bảo đảm an toàn thông tin cá nhân, tuân thủ tiêu
chuẩn kỹ thuật).
#   chạy: conftest test <Dockerfile> -p policy/ --namespace dockerfile
package dockerfile

import rego.v1

# Danh sách cổng quản trị/CSDL — giữ theo Appendix A.1 của lab (SSH, telnet, Docker daemon,
MySQL, Postgres, Redis, Mongo).
forbidden_ports := {"22", "23", "2375", "2376", "3306", "5432", "6379", "27017"}

required_labels := {"org.opencontainers.image.source", "org.opencontainers.image.revision"}

root_names := {"root", "0", "root:root", "0:0"}

# Chỉ STAGE CUỐI tạo ra image chạy thật; USER ở stage builder không áp dụng cho runtime.
final_stage := max([ins.Stage | some ins in input])

final_users := [ins.Value[0] | some ins in input; ins.Stage == final_stage; ins.Cmd ==
"user"]

last_user := final_users[count(final_users) - 1] if count(final_users) > 0

# LEAST-PRIVILEGE ------------------------------------------------------------
deny contains msg if {
```

```
        count(final_users) == 0
        msg := "LEAST-PRIVILEGE: final stage has no USER instruction; container runs as
root. (CIS Docker 4.1; PCI-DSS 2.2.6; NIST SP 800-190; NĐ 13/2023 Đ.26; Luật ATTTM 2015
Đ.19)"
}

deny contains msg if {
        last_user in root_names
        msg := sprintf("LEAST-PRIVILEGE: final USER '%s' is root. (CIS Docker 4.1; PCI-DSS
2.2.6; NĐ 13/2023 Đ.26)", [last_user])
}

warn contains msg if {
        last_user
        not last_user in root_names
        not regex.match(`^[0-9]+(:[0-9]+)?$`, last_user)
        msg := sprintf("LEAST-PRIVILEGE: final USER '%s' is not a fixed numeric UID. (CIS
Docker 4.1)", [last_user])
}

# ATTACK-SURFACE (firewall / RBAC) ----------------------------------------
port_number(raw) := split(raw, "/")[0] # bỏ hậu tố /tcp, /udp

deny contains msg if {
        some ins in input
        ins.Cmd == "expose"
        some raw in ins.Value
        port_number(raw) in forbidden_ports
        msg := sprintf("ATTACK-SURFACE: EXPOSE '%s' covers administrative/database port %s.
(CIS Docker 5.8; NIST SP 800-190; Luật ATTTM 2015 Đ.19)", [raw, port_number(raw)])
}

# SUPPLY-CHAIN — provenance khai báo (traceability; không thay cho chữ ký/attestation) --
label_blob := concat(" ", [concat(" ", ins.Value) | some ins in input; ins.Cmd == "label"])

deny contains msg if {
        some req in required_labels
        not contains(label_blob, req)
        msg := sprintf("SUPPLY-CHAIN: missing provenance label '%s' (declared provenance
for traceability). (OCI image-spec; SLSA Provenance L1; NIST SSDF)", [req])
}

# SUPPLY-CHAIN — base image (không :latest/ADD, ghim digest) -----------------
deny contains msg if {
        some ins in input
        ins.Cmd == "from"
        endswith(ins.Value[0], ":latest")
        msg := sprintf("SUPPLY-CHAIN: base image '%s' uses :latest; pin a version or
digest. (CIS Docker 4.5; SLSA)", [ins.Value[0]])
}

deny contains msg if {
        some ins in input
        ins.Cmd == "add"
        msg := "SUPPLY-CHAIN: use COPY instead of ADD to avoid remote/implicit extraction.
(CIS Docker 4.9)"
```

```
}

warn contains msg if {
        some ins in input
        ins.Cmd == "from"
        not contains(ins.Value[0], "@sha256:")
        not endswith(ins.Value[0], ":latest")
        msg := sprintf("SUPPLY-CHAIN: base image '%s' is not pinned by digest. (CIS Docker
4.5; SLSA)", [ins.Value[0]])
}

# SECRET-IN-IMAGE — cải tiến của nhóm (ngoài bản mẫu lab) --------------------
# Bí mật đặt trong ENV/ARG bị bake vào layer ảnh và lộ qua `docker history`. Ánh xạ
# luật secret của compose (F03) xuống tầng Dockerfile. ENV parse thành [key, value, "=",
...];
# ARG parse thành ["KEY=VALUE"].
secret_key := `(?i)(password|passwd|secret|token|api[_-]?key|access[_-]?key|private[_-]?
key|credential)`

deny contains msg if {
        some ins in input
        ins.Cmd == "env"
        some i, key in ins.Value
        i % 3 == 0
        i + 1 < count(ins.Value)
        regex.match(secret_key, key)
        ins.Value[i + 1] != ""
        msg := sprintf("SECRET-IN-IMAGE: ENV '%s' hardcodes a credential into the image
(exposed via docker history); use build secrets/runtime env. (CIS Docker 4.10; GDPR Art.32;
PCI-DSS 8.3; NĐ 13/2023 Đ.26)", [key])
}

deny contains msg if {
        some ins in input
        ins.Cmd == "arg"
        some tok in ins.Value
        contains(tok, "=")
        regex.match(secret_key, split(tok, "=")[0])
        split(tok, "=")[1] != ""
        msg := sprintf("SECRET-IN-IMAGE: ARG '%s' bakes a credential default into the
image. (CIS Docker 4.10; GDPR Art.32; PCI-DSS 8.3; NĐ 13/2023 Đ.26)", [split(tok, "=")[0]])
}
```

Phụ lục B — Mã nguồn chính sách docker-compose (compose.rego)

Nguồn: nhóm tự viết cho docker-compose — lab chỉ enforce trên manifest Kubernetes, nhóm chuyển các ý tưởng least-privilege/firewall/secret sang định dạng triển khai thật. Namespace `compose`, 5 luật `deny` (chặn backend mở cổng host, bind ngoài loopback, privileged, mount docker.sock, secret nội tuyến); bộ kiểm thử `compose_test.rego` có 6 ca. Thông báo vi phạm trích kèm điều khoản chuẩn quốc tế và Việt Nam (NĐ 13/2023 Đ.26; Luật ATTTM 2015 Đ.19). Chạy: `conftest test docker-compose.yml -p juice-shop/policy --namespace compose`.

```
package compose

import rego.v1

# Input: file docker-compose (conftest tự đọc YAML).
# ports chấp nhận cả 3 dạng của Compose: "127.0.0.1:3102:8080", 3000, {target, published,
host_ip}.
# Thông báo vi phạm trích điều khoản chuẩn quốc tế (GDPR, PCI-DSS, CIS Docker) và Việt Nam
# (Nghị định 13/2023/NĐ-CP Điều 26; Luật An toàn thông tin mạng 2015 Điều 19).

host_ip(p) := split(p, ":")[0] if {
        is_string(p)
        count(split(p, ":")) == 3
}

host_ip(p) := "" if {
        is_string(p)
        count(split(p, ":")) < 3
}

host_ip(p) := "" if is_number(p)

host_ip(p) := object.get(p, "host_ip", "") if is_object(p)

ports(svc) := object.get(svc, "ports", [])

# Dịch vụ được thiết kế để nhận kết nối từ host (reverse proxy, dashboard) phải tự khai báo
nhãn.
edge(svc) if object.get(object.get(svc, "labels", {}), "lab.exposure", "") == "edge"

# F03 / NET-02: backend không được publish cổng. Nếu backend có cổng host, mọi route (kể cả
/metrics)
# có thể bị truy cập vòng qua proxy, làm vô hiệu luật chặn /metrics và giới hạn tốc độ ở
proxy.
deny contains msg if {
        some name, svc in input.services
        count(ports(svc)) > 0
        not edge(svc)
        msg := sprintf("[F03 NET-02] service '%s' publishes ports but is not an edge
```

```
service (label lab.exposure=edge); backend must only be reachable through the proxy (Luật
ATTTM 2015 Đ.19; NĐ 13/2023 Đ.26)", [name])
}

# NET-01 (GDPR Art.32): cổng publish chỉ được bind loopback trong lab.
deny contains msg if {
        some name, svc in input.services
        some p in ports(svc)
        host_ip(p) != "127.0.0.1"
        msg := sprintf("[GDPR Art.32 NET-01] service '%s' publishes %v on all interfaces
(NĐ 13/2023 Đ.26; Luật ATTTM 2015 Đ.19)", [name, p])
}

deny contains msg if {
        some name, svc in input.services
        svc.privileged == true
        msg := sprintf("[CIS Docker 5.4] service '%s' runs privileged", [name])
}

deny contains msg if {
        some name, svc in input.services
        some v in object.get(svc, "volumes", [])
        contains(sprintf("%v", [v]), "docker.sock")
        msg := sprintf("[CIS Docker 5.31] service '%s' mounts the Docker socket", [name])
}

# Biến môi trường có thể là danh sách "KEY=value" hoặc map {KEY: value}; *_FILE trỏ tới
secret thì được phép.
env(svc) := [e | some e in svc.environment] if is_array(svc.environment)

env(svc) := [sprintf("%s=%v", [k, v]) | some k, v in svc.environment] if
is_object(svc.environment)

env(svc) := [] if not svc.environment

deny contains msg if {
        some name, svc in input.services
        some e in env(svc)
        regex.match(`(?i)^[A-Z0-9_]*(PASSWORD|SECRET|TOKEN|API_KEY)=.+`, e)
        msg := sprintf("[PCI-DSS 8.3 / GDPR Art.32 / NĐ 13/2023 Đ.26] service '%s' has an
inline secret in environment: %s", [name, split(e, "=")[0]])
}

drops_all_caps(svc) if "ALL" in object.get(svc, "cap_drop", [])

warn contains msg if {
        some name, svc in input.services
        not drops_all_caps(svc)
        msg := sprintf("[CIS Docker 5.3] service '%s' does not drop capabilities", [name])
}

warn contains msg if {
        some name, svc in input.services
        not svc.read_only
        msg := sprintf("[CIS Docker 5.12] service '%s' has a writable root filesystem",
```

```
[name])
}
```